Sprawozdanie 4

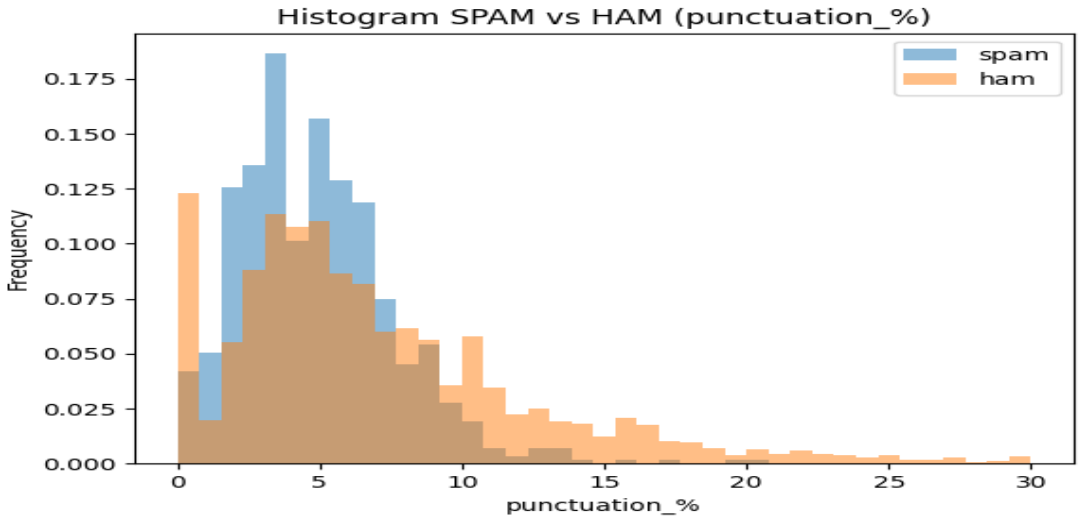
Kacper Dusza, 131427, INFORMATYKA 3 ROK

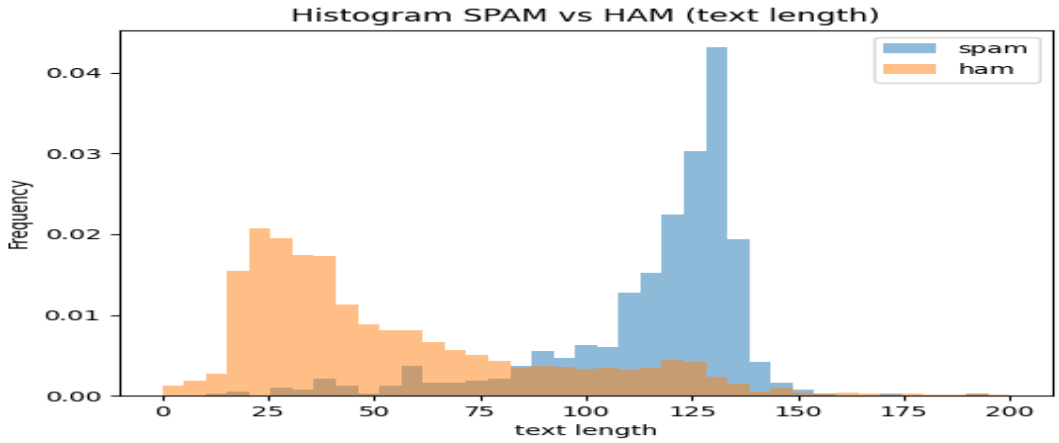
Zd.1

ex1.py

```python
import string
import numpy as np
import pandas as pd
from matplotlib import pyplot

pd.set_option( pat: 'display.max_columns', val: None)

data = pd.read_csv( filepath_or_buffer: 'SMSSpamCollection.tsv', sep='\t')
data.columns = ['label', 'text']

def count_punctuation_percent(text):  1 usage
    count = sum([1 for char in text if char in string.punctuation])
    no_space = len(text) - text.count(' ')

    return round(count / no_space, 3) * 100

data["text_length"] = data["text"].apply(lambda x: len(x) - x.count(' '))
data["punctuation_%"] = data["text"].apply(lambda x: count_punctuation_percent(x))

#print(data.head())
```

```python
bins_text_length = np.linspace( start: 0,  stop: 200,  num: 40)

pyplot.hist(data[data['label'] == 'spam']["text_length"], bins_text_length, alpha=0.5, density=True, label="spam")
pyplot.hist(data[data['label'] == 'ham']["text_length"], bins_text_length, alpha=0.5, density=True, label="ham")

pyplot.title("Histogram SPAM vs HAM (text length)")
pyplot.xlabel("text length")
pyplot.ylabel("Frequency")
pyplot.legend(loc= 'upper right')

pyplot.savefig("text_length.png")
pyplot.show()

bins_punct_percent = np.linspace( start: 0,  stop: 30,  num: 40)

pyplot.hist(data[data['label'] == 'spam']["punctuation_%"], bins_punct_percent, alpha=0.5, density=True, label="spam")
pyplot.hist(data[data['label'] == 'ham']["punctuation_%"], bins_punct_percent, alpha=0.5, density=True, label="ham")

pyplot.title("Histogram SPAM vs HAM (punctuation_%)")
pyplot.xlabel("punctuation_%")
pyplot.ylabel("Frequency")

pyplot.legend(loc= 'upper right')
pyplot.savefig("punctuation_%.png")
pyplot.show()
```

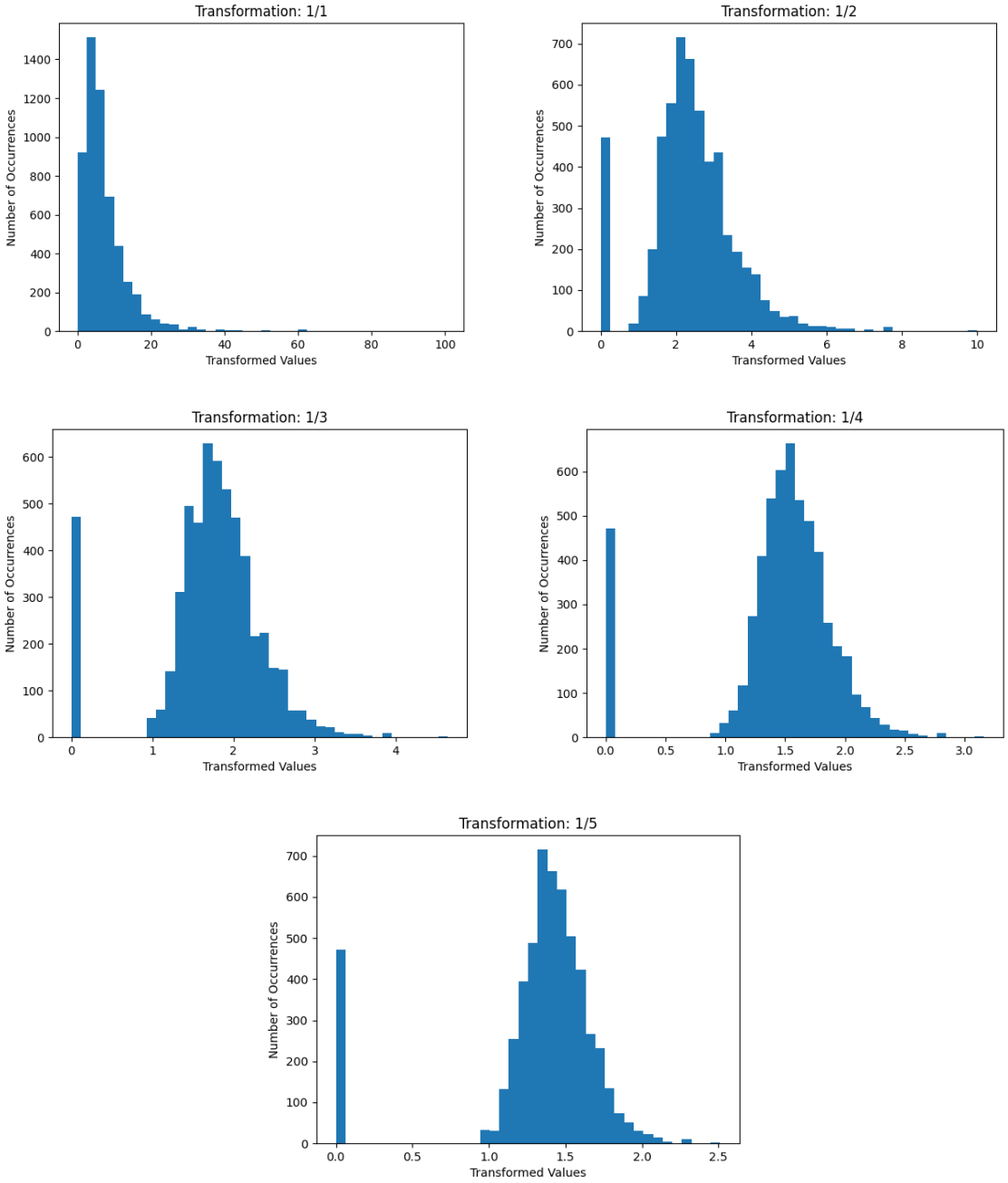




Cecha długości tekstu wyraźnie pokazuje nam zależność, która polega na częstszym przypadku spamu przy dłuższych wiadomościach (~115+ znaków). Cechę procentowej zawartości należy przetransformować.

W tym wypadku najlepszą transformacją jest transformacja po 4 i 5 iteracji, ponieważ wykres bardziej przypomina krzywą dzwonową co stanowi o większej stabilności modelu.

```
for i in [1,2,3,4,5]:
    trans_data = data["punctuation_%"] ** (1/i)

    pyplot.hist(trans_data, bins = 40)

    pyplot.title(f"Transformation: 1/{i}")
    pyplot.xlabel("Transformed Values")
    pyplot.ylabel("Number of Occurrences")

    pyplot.savefig(f"transformation_1_{i}.png")
    pyplot.show()

# W tym wypadku najlepszą transformacją jest transofrmacja po 4 i 5 iteracji,
# ponieważ wykres bardziej przypomina krzywą dzwonową
# co stanowi o większej stabnilności modelu.
```


Transformation: 1/1
Number of Occurrences
Transformed Values
Transformation: 1/2
Number of Occurrences
Transformed Values
Transformation: 1/3
Number of Occurrences
Transformed Values
Transformation: 1/4
Number of Occurrences
Transformed Values
Transformation: 1/5
Number of Occurrences
Transformed Values

Zd.2

Słupek po lewej stronie pojawiający się na wykresach, gdzie i > 2 reprezentuje wiadomości, które nie zawierają żadnych znaków interpunkcyjnych. Wadą transformacji Boxa Coxa jest brak możliwości normalizacji wiadomości nieposiadających znaków interpunkcyjnych. Transformacja polega na podnoszeniu wartości do danej potęgi, jednak 0 podniesione do jakiejkolwiek potęgi zawsze wynosi 0. Stąd charakterystyczny słupek po lewej stronie wykresów.